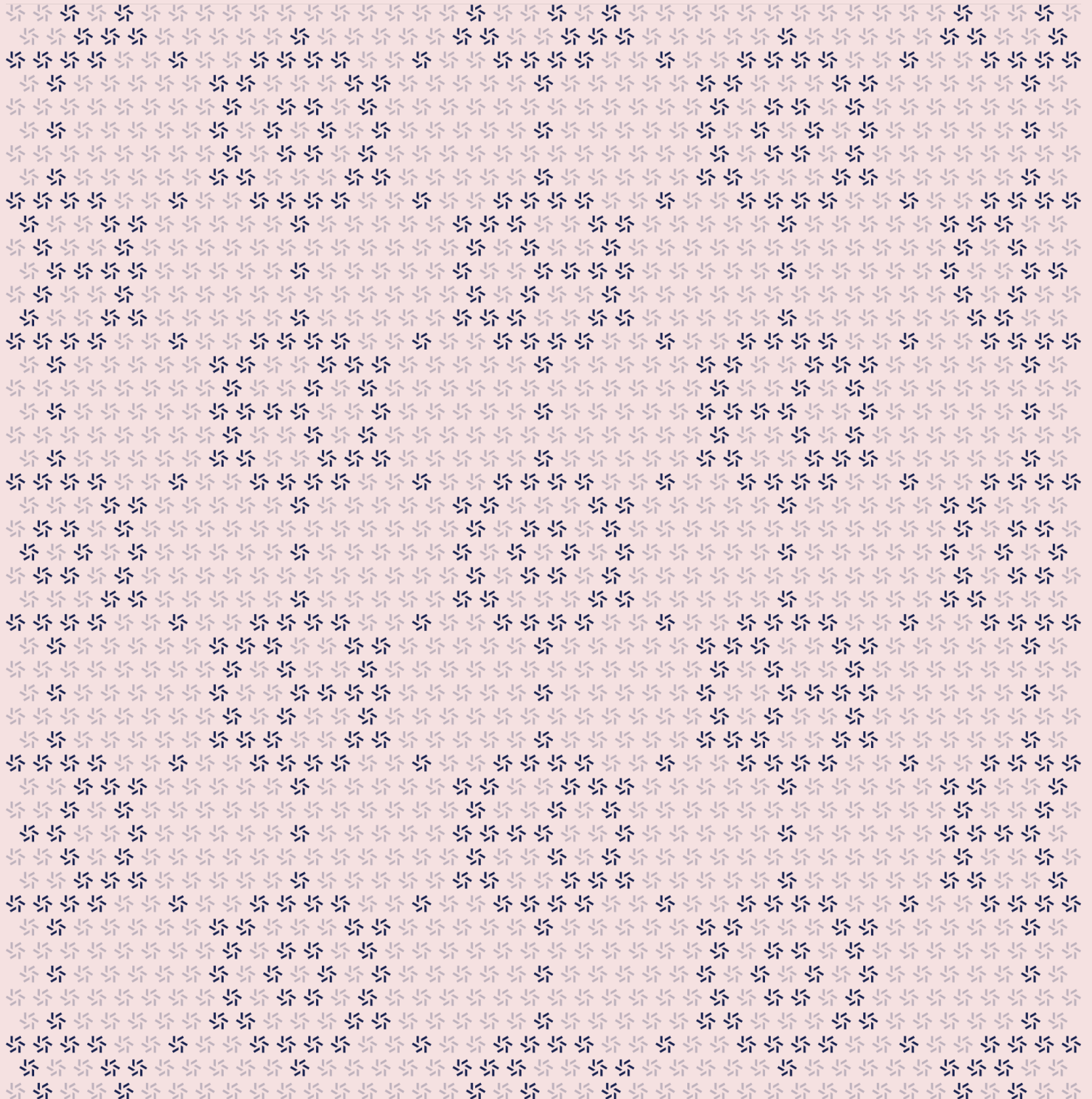


February 11, 2026

Starkware (ERC20)

Smart Contract Security Assessment



Contents

About Zellic	4
1. Overview	4
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	6
2.1. About Starkware (ERC20)	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	10
3.1. Upper 128 bits of the amount are truncated in lock_and_delegate_message_hash	11
4. System Design	13
4.1. Contract: ERC20Lockable	14
4.2. Contract: ERC20Mintable	14
4.3. File: eip712_utils.cairo	14

5.	Assessment Results	14
5.1.	Disclaimer	15

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Starkware from February 5th to February 6th, 2026. During this engagement, Zellic reviewed Starkware (ERC20)'s code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Does the scoped code comply with the [SNIP-2](#) standard?
 - Are there any security bugs?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Starkware (ERC20) files, we discovered one finding, which was informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
 Critical	0
 High	0
 Medium	0
 Low	0
 Informational	1

2. Introduction

2.1. About Starkware (ERC20)

Starkware contributed the following description of Starkware (ERC20):

StarkGate tokens are ERC-20 style token contracts written in Cairo for Starknet. This audit is aimed to validate a number of features and changes to these contracts.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the files.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Starkware (ERC20) Files

Type	cairo
Platform	Cairo-compatible
Target	starkgate-contracts
Repository	https://github.com/starknet-io/starkgate-contracts ↗
Version	c763c1f74692fbdd5015e023e765c403bfc01b95
Programs	strk/src/erc20_lockable.cairo strk/src/eip712_utils.cairo sg_token/src/erc20_mintable.cairo

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of four person-days. The assessment was conducted by two consultants over the course of two calendar days.

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
✈ Engagement Manager
pedro@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Jinseo Kim
✈ Engineer
jinseo@zellic.io ↗

Quentin Lemauf
✈ Engineer
quentin@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

February 5, 2026 Kick-off call

February 5, 2026 Start of primary review period

February 6, 2026 End of primary review period

3. Detailed Findings

3.1. Upper 128 bits of the amount are truncated in lock_and_delegate_message_hash

Target	eip712_utils.cairo		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

A user can lock and delegate their tokens. They can directly invoke the lock_and_delegate function in ERC20Lockable function, or they can sign their request and let anyone to invoke the lock_and_delegate function with the signature.

Internally, the lock_and_delegate function receives the parameters and the signature – the lock_and_delegate_message_hash is responsible to calculate the hash of the payload to be signed:

```
fn lock_and_delegate_by_sig(
    ref self: ContractState,
    account: ContractAddress,
    delegatee: ContractAddress,
    amount: u256,
    nonce: felt252,
    expiry: u64,
    signature: Array<felt252>,
) {
    // (...)
    let hash = lock_and_delegate_message_hash(
        :domain, :account, :delegatee, :amount, :nonce, :expiry,
    );
    // (...)
    validate_signature(:account, :hash, :signature);
    // (...)
}
```

```
/// Calculates the message hash for signing, following the SNIP equivalent of
/// EIP-712.
pub fn lock_and_delegate_message_hash(
    domain: felt252,
    account: ContractAddress,
    delegatee: ContractAddress,
```

```

        amount: u256,
        nonce: felt252,
        expiry: u64,
    ) -> felt252 {
        let input_hash = lock_and_delegate_input_hash(:delegatee, :amount, :nonce, :expiry);
        let message_inputs = array![STARKNET_MESSAGE, domain, account.into(), input_hash].span();
        pedersen_hash_span(message_inputs)
    }

    /// Calculates the hash of lock and delegate input parameters.
    fn lock_and_delegate_input_hash(
        delegatee: ContractAddress, amount: u256, nonce: felt252, expiry: u64,
    ) -> felt252 {
        let lock_and_delegate_inputs = array![
            LOCK_AND_DELEGATE_TYPE_HASH, delegatee.into(), amount.low.into(),
            nonce, expiry.into(),
        ]
        .span();
        pedersen_hash_span(lock_and_delegate_inputs)
    }

```

Note that the `lock_and_delegate_input_hash` function only uses the low part of the amount parameter. The high part, which stores the upper 128 bits of the amount, will be discarded. Hence, the hashes for the amount x and $x + 2^{128}$ would be equivalent provided that other parameters are unchanged.

Impact

This should not affect the business logic of the project practically, since the `lock_and_delegate_by_sig` requires the account to hold at least the specified amount of tokens. STRK token, which is planned to be locked through this contract, has the total supply of 10^{28} .

Nevertheless, we still recommend to implement a fix considering that this code may be reused elsewhere and the behavior can be confusing to developers.

Recommendations

Consider ensuring that the high part of the amount is equal to zero. Alternatively, consider revising the signature payload format to support u256 parameter.

Remediation

This issue has been acknowledged by Starkware.

4. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

4.1. Contract: ERC20Lockable

The scope includes ERC20Lockable, a lockable token contract that adds signature-based lock-and-delegate behavior on top of permissioned minting and burning, role-based administration, and upgrade-delay controls. The in-scope logic primarily covers locking-contract configuration, direct and signature-based lock/delegate entry points, minter-gated supply changes, and the L1 rescue approval handler.

4.2. Contract: ERC20Mintable

The scope includes ERC20Mintable, which provides the baseline permissioned mint/burn token flow with role initialization, replaceability delay setup, allowance helper wrappers, and L1 rescue approval handling. Relative to the lockable variant, this contract has a narrower surface area focused on minter authorization and standard token-state updates.

4.3. File: eip712_utils.cairo

The scope includes eip712_utils.cairo, which supplies the shared signature-validation and message-hashing primitives used by lock-and-delegate requests and follows the SNIP-2 typed-message pattern. The relevant logic covers domain hashing, request-hash construction, and account-contract signature checks, and its correctness depends on consistent off-chain and on-chain encoding assumptions.

5. Assessment Results

During our assessment on the scoped Starkware (ERC20) files, we discovered one finding, which was informational in nature.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.